eGOV APIS

Back to dashboard

API DOCUMENTATION

#NationalID | eVerify

Verify citizen identity against PhilSys in real time, with consent built into every check.

API documentation | Integration | Credentials | Usage

Face Liveness Web SDK Integration Guide

The eVerify Face Liveness Web SDK allows Relying Parties (RPs) to perform biometric face liveness checks on client applications. A successful liveness check generates a session ID and a temporary URL of the captured face image, which are then passed to the eVerify backend query/verification endpoints.

1. Import the SDK

Add the official eVerify Face Liveness Web SDK script tag to your web application's HTML structure (e.g., inside the `<head>` or `<body>` tag).

```
<script src="https://hackathon-everify-face-liveness.e.gov.ph/js/everify-liveness-sdk.min.js"><
```

2. Initialize and Start the Check

Call the `window.eKYC().start()` promise function, passing your client application's Public API Key to initialize the verification window.

```
window.eKYC().start({
    pubKey: "YOUR_PUBLIC_API_KEY"
})
.then((response) => {
    // Face liveness verification completed successfully
    console.log("Liveness check response:", response);

    const sessionId = response.result.session_id;
    const photoUrl = response.result.photo_url;
```

```
    // Now send sessionId or photoUrl to your backend to verify against demographics or QR code
})
.catch((error) => {
    // Handle error or user cancellation
    console.error("Liveness check error or cancelled:", error);
});
```

3. SDK Response Payload

On success, the SDK resolves with the following JSON structure:

```
{
    "status": "COMPLETED",
    "result": {
        "photo": "...",
        "session_id": "a1b3fae6-af74-4896-bd58-32a81604de01",
        "photo_url": "https://liveness.photo.url/image.jpg?expires=123"
    }
}
```

Property Description

Property	Type	Description
`status`	string	The status of the session. E.g., `COMPLETED`.
`result.photo`	string	A base64-encoded string representing the captured selfie.
`result.session_id`	string (UUID)	The unique identifier for the completed liveness session.
`result.photo_url`	string	A secure temporary URL pointing to the captured face image.

4. Submitting Biometrics to eVerify API

Once your frontend has obtained the `session_id`, you should send it to your backend. Your backend will then call the eVerify query/verification endpoints.

Pass the `face_liveness_session_id` directly in the payload:

```
{
    "first_name": "Juan",
    "middle_name": "Santos",
    "last_name": "Dela Cruz",
    "suffix": "JR",
    "birth_date": "1989-09-12",
```

```
    "face_liveness_session_id": "a1b3fae6-af74-4896-bd58-32a81604de01"
}
```

5. Full HTML Integration Example

Here is a minimal HTML file demonstrating the complete integration of the Face Liveness Web SDK:

```
<!DOCTYPE html>
<html>
<head>
    <title>eVerify Face Liveness Integration</title>
    <!-- Load the official eVerify Face Liveness Web SDK -->
    <script src="https://hackathon-everify-face-liveness.e.gov.ph/js/everify-liveness-sdk.min.j
</head>
<body>

    <button onclick="startVerification()">Verify Identity</button>
    <pre id="output"></pre>

    <script>
        function startVerification() {
            window.eKYC().start({
                // Replace with your actual Public API Key
                pubKey: "YOUR_PUBLIC_API_KEY"
            })
            .then((response) => {
                document.getElementById('output').textContent = JSON.stringify(response, null,
            })
            .catch((error) => {
                document.getElementById('output').textContent = JSON.stringify(error, null, 2)
            });
        }
    </script>

</body>
</html>
```